MedPilot

OpenMRS Agentic Clinical Copilot Platform

Software Requirements Specification (SRS) — Version 2.0

Document Version	2.0 — Full CRUD API Integration Release
Classification	Confidential — Internal Use Only
Document Status	Draft for Review
Prepared By	MedPilot Product & Engineering Team
Date	April 2026
Standard	ISO/IEC 29148:2018 — Requirements Engineering
API Standard	HL7 FHIR R4 + OpenMRS REST API v1
Key Change from v1.0	Full patient lifecycle: CREATE, READ, UPDATE, DELETE across all clinical entities including patient registration, PDF/EHR ingestion, and Health Gorilla integration

⚕ Intended Use Statement

MedPilot is a clinical workflow assistance platform — NOT a medical device, diagnostic tool, or autonomous prescribing system. All clinical decisions remain the responsibility of licensed healthcare professionals. MedPilot augments — but does not replace — clinical judgment.

© 2026 MedPilot. Proprietary and Confidential.

1. Introduction & Scope

1.1 Purpose

This Software Requirements Specification (SRS) v2.0 defines the complete functional, integration, safety, and non-functional requirements for MedPilot — an agentic AI clinical copilot for OpenMRS. This version supersedes v1.0 and incorporates full patient lifecycle management (CREATE, READ, UPDATE, DELETE), PDF/EHR record ingestion, and Health Gorilla external lab integration based on the delivered API implementation files.

1.2 What Changed in v2.0

New in v2.0 — Scope Expansions

- ✅ Patient Registration — createPatient via FHIR + REST API (was missing in v1.0)
- ✅ Full DELETE support — AllergyIntolerance, Condition, Observation (was undefined in v1.0)
- ✅ Full UPDATE support — Allergy (PATCH), Condition (PATCH), Observation (PUT), Medication (PATCH)
- ✅ PDF/EHR Ingestion — parse and ingest patient record PDFs into OpenMRS
- ✅ Health Gorilla Integration — retrieve patients + conditions and sync to OpenMRS
- ✅ Vital Signs — Height, Weight, Temperature, Respiratory Rate, SpO2 (CREATE + UPDATE)
- ✅ MedicationDispense — full CREATE and GET support
- ✅ Duplicate detection — allergy and medication deduplication before write

1.3 Product Vision

"A full-lifecycle clinical workflow copilot for OpenMRS — enabling natural language patient registration, data ingestion, querying, updating, and deletion across all clinical entities."

2. Complete OpenMRS API Catalogue

The following tables document every API operation MedPilot MUST support, derived directly from the FHIR LAB Postman collection, EHR_script.py, and Health Gorilla integration scripts included in the delivered codebase.

2.1 Patient APIs

Method	Endpoint	Description	Key Parameters / Body Fields
GET	`/ws/fhir2/R4/Patient?identifier={id}`	Resolve patient UUID from OpenMRS identifier	`identifier (e.g. 10001YY)`
GET	`/ws/rest/v1/patient?q={query}`	Full-text patient search by name or ID	`q (search string)`
POST	`/ws/rest/v1/patient`	Create new patient with demographics and identifier	`person{names,gender,birthdate,addresses}, identifiers[]`
GET	`/ws/fhir2/R4/Patient/{uuid}`	Retrieve full patient FHIR resource by UUID	`uuid (path param)`

2.2 Encounter APIs

Method	Endpoint	Description	Key Parameters / Body Fields
POST	/ws/fhir2/R4/Encounter	Create a new clinical encounter (FHIR)	resourceType, status, class, subject, period, participant, location
POST	/ws/rest/v1/encounter	Create encounter via REST (with role/provider UUIDs)	encounterDatetime, patient, encounterType, location, encounterProviders[]
GET	/ws/fhir2/R4/Encounter?patient={uuid}	Retrieve all encounters for a patient	patient (UUID)

2.3 Observation APIs

Observations cover all vital signs and clinical measurements. The system supports CREATE (POST) and full UPDATE (PUT) per the delivered Postman collection.

Method	Endpoint	Description	Key Parameters / Body Fields
GET	/ws/fhir2/R4/Observation?patient={uuid}	Retrieve all observations for a patient	patient (UUID)
GET	/ws/fhir2/R4/Observation/{uuid}	Retrieve single observation by UUID	uuid (path param)
POST	/ws/fhir2/R4/Observation	Create observation — Height, Weight, Temp, RR, SpO2	resourceType, status, code{coding[]}, subject, effectiveDateTime, valueQuantity{value,unit}
PUT	/ws/fhir2/R4/Observation/{uuid}	Full replacement update of an existing observation	Full Observation FHIR resource with id field
DELETE	/ws/fhir2/R4/Observation/{uuid}	Delete an observation record	uuid (path param)
POST	/ws/rest/v1/obs	Create observation via REST (concept UUID lookup)	person, concept, obsDatetime, value

2.3.1 Supported Vital Sign Concepts (CIEL Codes)

Vital Sign	CIEL Code	FHIR Code	Unit
Height	5090AAAA...	5090	cm
Weight	5089AAAA...	5089	kg
Temperature	5088AAAA...	5088	°C
Respiratory Rate	5242AAAA...	5242	breaths/min
Oxygen Saturation (SpO_2)	5092AAAA...	5092	%
Systolic Blood Pressure	5085AAAA...	5085	mmHg
Diastolic Blood Pressure	5086AAAA...	5086	mmHg

© 2026 MedPilot. Proprietary and Confidential.

2.4 Condition APIs

Method	Endpoint	Description	Key Parameters / Body Fields
GET	/ws/fhir2/R4/Condition?patient={uuid}	Retrieve all conditions for a patient	patient (UUID)
POST	/ws/fhir2/R4/Condition	Create a new condition/diagnosis	resourceType, clinicalStatus, verificationStatus, category, code{coding[]}, subject, onsetDateTime
POST	/ws/rest/v1/condition	Create condition via REST	patient, condition{coded:uuid}, clinicalStatus, verificationStatus, onsetDate
PATCH	/ws/fhir2/R4/Condition/{uuid}	Partial update — e.g., change clinical status	JSON Patch: [{op:'replace', path:'/clinicalStatus/coding/0/code', value:'inactive'}]
DELETE	/ws/fhir2/R4/Condition/{uuid}	Delete a condition record	uuid (path param)

2.5 Allergy APIs

Method	Endpoint	Description	Key Parameters / Body Fields
GET	/ws/fhir2/R4/AllergyIntolerance?patient={uuid}	Retrieve all allergies for a patient	patient (UUID)
POST	/ws/fhir2/R4/AllergyIntolerance	Create a new allergy record	resourceType, clinicalStatus, verificationStatus, type, category[], criticality, code{coding[]}, patient, reaction[{substance,manifestation[],severity}]
PATCH	/ws/fhir2/R4/AllergyIntolerance/{uuid}	Partial update — e.g., change reaction severity	JSON Patch: [{op:'replace', path:'/reaction/0/severity', value:'severe'}]
DELETE	/ws/fhir2/R4/AllergyIntolerance/{uuid}	Delete an allergy record	uuid (path param)
GET	/ws/rest/v1/patient/{uuid}/allergy	Get patient allergies via REST (for dedup check)	patient uuid (path param)
POST	/ws/rest/v1/patient/{uuid}/allergy	Create allergy via REST with concept UUIDs	allergen{allergenType, codedAllergen{uuid}}, severity{uuid}, reactions[], comment

2.6 Medication APIs

Method	Endpoint	Description	Key Parameters / Body Fields
GET	/ws/fhir2/R4/MedicationRequest?patient={uuid}	Retrieve all medication orders for a patient	patient (UUID)
PATCH	/ws/fhir2/R4/MedicationRequest/{uuid}	Update medication order — e.g., stop medication	JSON Patch: [{op:'replace', path:'/status', value:'stopped'}]
POST	/ws/rest/v1/order	Create drug order via REST	type:'drugorder', patient, drug, concept, dose, doseUnits, route, frequency, duration, quantity, careSetting, encounter, orderer
GET	/ws/rest/v1/order?patient={uuid}	Check existing orders for deduplication	patient (UUID)
POST	/ws/fhir2/R4/MedicationDispense	Record medication dispensing event	resourceType, status:'completed', subject, medicationReference, quantity{value,unit}, whenHandedOver, dosageInstruction[]
GET	/ws/fhir2/R4/MedicationDispense?patient={uuid}	Retrieve dispensing records for a patient	patient (UUID)

2.7 Utility & Metadata APIs

Method	Endpoint	Description	Key Parameters / Body Fields
GET	/ws/fhir2/R4/metadata	FHIR CapabilityStatement — detect supported resources	(none)
GET	/ws/rest/v1/concept?q={name}	Resolve concept UUID by name (CIEL lookup)	q (concept name)
GET	/ws/rest/v1/drug?q={name}	Resolve drug UUID by name	q (drug name)
GET	/ws/rest/v1/encountertype?q={name}	Resolve encounter type UUID	q (type name)
GET	/ws/rest/v1/location?q={name}	Resolve location UUID	q (location name)
GET	/ws/rest/v1/provider?q={name}	Resolve provider UUID	q (provider name)
GET	/ws/rest/v1/encounterrole?q={name}	Resolve encounter role UUID	q (role name)
GET	/ws/rest/v1/caresetting?q={name}	Resolve care setting UUID	q (setting name)
POST	/ws/rest/v1/concept	Create new concept (for non-coded HG conditions)	names[], datatype, conceptClass, descriptions[]

© 2026 MedPilot. Proprietary and Confidential.

3. Patient Registration & Data Ingestion

3.1 New Patient Registration

The chatbot MUST support natural language commands to register a new patient in OpenMRS. The agent resolves the request to the createPatient tool, constructs the FHIR-compatible payload, presents a confirmation preview, and submits to the REST API on confirmation.

```
// createPatient payload (POST /ws/rest/v1/patient)
{
  "person": {
    "names": [{ "givenName": "Maria", "familyName": "Santos" }],
    "gender": "F",
    "birthdate": "1985-03-22",
    "addresses": [{ "address1": "123 Main St", "cityVillage": "Nairobi", "country":
"Kenya" }]
  },
  "identifiers": [{
    "identifier": "10023AF",
    "identifierType": "05a29f94-c0ed-11e2-94be-8c13b969e334",
    "location": "8d6c993e-c2cc-11de-8d13-0010c6dffd0f",
    "preferred": true
  }]
}
```

REQ-ID	Requirement	Priority	Category
FR-REG-01	System MUST accept a natural language prompt to register a new patient (e.g., 'Add patient John Doe, male, born 1990-05-12')	MUST	Registration
FR-REG-02	System MUST auto-generate a valid OpenMRS identifier using the Luhn-variant check digit algorithm	MUST	Registration
FR-REG-03	System MUST present a full registration preview (name, gender, DOB, address, identifier) before creating the record	MUST	Write Safety
FR-REG-04	System MUST detect and warn on potential duplicate patients (same name + DOB) before proceeding	MUST	Data Quality
FR-REG-05	System SHOULD support bulk patient registration from a CSV file (First Name, Last Name, Birthdate, Gender)	SHOULD	Ingestion

3.2 PDF / EHR Record Ingestion

MedPilot MUST support ingestion of structured patient record PDFs. The system parses the PDF using the pdfplumber-based parser (parsing.py) to extract demographics, conditions, observations, allergies, and medications, then ingests each entity into OpenMRS via the appropriate API.

PDF Section	Extracted Fields	OpenMRS API Target
Patient Demographics	Name, Age, Gender, Encounter Type, Location, Provider, Encounter Role	`/ws/rest/v1/encounter (create encounter context)`
Problem List	Condition name, clinical status, verification status, onset date	`/ws/rest/v1/condition (POST per condition)`
Vital Signs	Weight (kg), Height (cm), Systolic BP, Diastolic BP	`/ws/fhir2/R4/Observation (POST per vital)`

© 2026 MedPilot. Proprietary and Confidential.

PDF Section	Extracted Fields	OpenMRS API Target
Allergies	Allergen name, severity, reaction type, comment	`/ws/rest/v1/patient/{uuid}/allergy (POST per allergy)`
Medications	Drug name, dose, route, frequency, duration, quantity, refills, orderer	`/ws/rest/v1/order (POST per medication)`

REQ-ID	Requirement	Priority	Category
FR-ING-01	System MUST accept PDF upload and parse it using the structured section parser (Patient Demographics, Problem List, Vital Signs, Allergies, Medications)	MUST	Ingestion
FR-ING-02	System MUST map all parsed sections to the corresponding OpenMRS API calls and execute with confirmation	MUST	Ingestion
FR-ING-03	System MUST check for duplicate allergies and medications before writing (skip if already present)	MUST	Data Quality
FR-ING-04	System MUST report a per-entity ingestion result (success / skipped / failed) after processing	MUST	Feedback
FR-ING-05	System MUST resolve concept, drug, provider, location UUIDs via lookup before constructing write payloads	MUST	Integration
FR-ING-06	System SHOULD support structured EHR data import from JSON payloads in addition to PDF	SHOULD	Ingestion

3.3 Health Gorilla External Lab Integration

MedPilot MUST support retrieval of patient records and conditions from Health Gorilla's FHIR sandbox and synchronisation into OpenMRS.

Method	Endpoint	Description	Key Parameters / Body Fields
GET	`https://sandbox.healthgorilla.com/fhir/Patient?given={}&family={}&birthdate={}`	Search for a patient in Health Gorilla by name and DOB	`given, family, birthdate`
GET	`https://sandbox.healthgorilla.com/fhir/Condition?patient={hg_id}`	Retrieve all conditions for a Health Gorilla patient	`patient (HG patient ID), Bearer token`

REQ-ID	Requirement	Priority	Category
FR-HG-01	System MUST authenticate to Health Gorilla using a Bearer token obtained from HealthGorillaTokenService	MUST	Integration
FR-HG-02	System MUST retrieve matching patients from Health Gorilla by first name, last name, and birthdate	MUST	Integration
FR-HG-03	System MUST retrieve and map Health Gorilla Condition resources to OpenMRS condition format	MUST	Integration
FR-HG-04	System MUST create non-coded conditions as new OpenMRS concepts when no matching concept UUID exists	MUST	Integration
FR-HG-05	System MUST support bulk patient sync from a CSV file (First Name, Last Name, Birthdate columns)	SHOULD	Integration

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
FR-HG-06	System MUST limit condition import to a configurable maximum per patient (default: 20) to prevent data flooding	MUST	Safety
FR-HG-07	System MUST present an import summary preview before committing any Health Gorilla data to OpenMRS	MUST	Write Safety

4. Complete Tool Registry

The following is the complete, authoritative tool registry for MedPilot v2.0. Every tool is backed by a specific OpenMRS API operation validated against the delivered codebase. Tools marked Write=YES require explicit user confirmation before execution.

4.1 Patient Tools

Tool Name	Write?	Description	API Backing	Permission
`searchPatients`	NO	Full-text and identifier-based patient search	`GET /ws/rest/v1/patient`	read:patient
`resolvePatient`	NO	Resolve patient UUID from name, ID, or session context	`GET /ws/fhir2/R4/Patient?identifier`	read:patient
`getPatientDemographics`	NO	Retrieve structured demographics for a patient	`GET /ws/fhir2/R4/Patient/{uuid}`	read:patient
`createPatient`	YES	Register a new patient with demographics and auto-generated identifier	`POST /ws/rest/v1/patient`	write:patient

4.2 Encounter Tools

Tool Name	Write?	Description	API Backing	Permission
`getEncounters`	NO	Retrieve encounter history for a patient	`GET /ws/fhir2/R4/Encounter?patient`	read:clinical
`createEncounter`	YES	Create a new clinical encounter with type, location, provider, role	`POST /ws/rest/v1/encounter`	write:encounter
`createEncounterFHIR`	YES	Create encounter via FHIR (status, class, period, participant)	`POST /ws/fhir2/R4/Encounter`	write:encounter

4.3 Observation Tools

Tool Name	Write?	Description	API Backing	Permission
`getObservations`	NO	Retrieve all observations for a patient	`GET /ws/fhir2/R4/Ob`	read:clinical

Tool Name	Write?	Description	API Backing	Permission
			`servation?patient`	
`getObservation`	NO	Retrieve a single observation by UUID	`GET /ws/fhir2/R4/Observation/{uuid}`	read:clinical
`createObservation`	YES	Create a vital sign or clinical observation (FHIR)	`POST /ws/fhir2/R4/Observation`	write:observation
`createObservationREST`	YES	Create observation via REST using concept UUID lookup	`POST /ws/rest/v1/obs`	write:observation
`updateObservation`	YES	Full replacement update of an existing observation (PUT)	`PUT /ws/fhir2/R4/Observation/{uuid}`	write:observation
`deleteObservation`	YES	Delete an observation record — requires explicit confirmation	`DELETE /ws/fhir2/R4/Observation/{uuid}`	write:observation

4.4 Condition Tools

Tool Name	Write?	Description	API Backing	Permission
`getConditions`	NO	Retrieve all conditions/diagnoses for a patient	`GET /ws/fhir2/R4/Condition?patient`	read:clinical
`createCondition`	YES	Add a new condition with status, verification, and onset date	`POST /ws/fhir2/R4/Condition`	write:condition
`updateCondition`	YES	Patch a condition — e.g., set clinical status to inactive	`PATCH /ws/fhir2/R4/Condition/{uuid}`	write:condition
`deleteCondition`	YES	Delete a condition record — requires explicit confirmation	`DELETE /ws/fhir2/R4/Condition/{uuid}`	write:condition

4.5 Allergy Tools

Tool Name	Write?	Description	API Backing	Permission
`getAllergies`	NO	Retrieve all allergies and adverse reactions for a patient	`GET /ws/fhir2/R4/AllergyIntolerance?patient`	read:clinical
`createAllergy`	YES	Create an allergy record with allergen, severity, reaction — dedup check first	`POST /ws/fhir2/R4/AllergyIntolerance`	write:allergy
`updateAllergy`	YES	Patch an allergy — e.g., update reaction severity	`PATCH /ws/fhir2/R4/AllergyIntolerance/{uuid}`	write:allergy

Tool Name	Write?	Description	API Backing	Permission
`deleteAllergy`	YES	Delete an allergy record — requires explicit confirmation	`DELETE /ws/fhir2/R4/AllergyIntolerance/{uuid}`	write:allergy

4.6 Medication Tools

Tool Name	Write?	Description	API Backing	Permission
`getMedications`	NO	Retrieve all active medication orders for a patient	`GET /ws/fhir2/R4/MedicationRequest?patient`	read:clinical
`checkMedicationExists`	NO	Check if a medication is already active (dedup before write)	`GET /ws/rest/v1/order?patient`	read:clinical
`createMedication`	YES	Create a drug order with dose, route, frequency, duration, quantity	`POST /ws/rest/v1/order`	write:medication
`updateMedication`	YES	Patch a medication request — e.g., stop an active medication	`PATCH /ws/fhir2/R4/MedicationRequest/{uuid}`	write:medication
`getMedicationDispense`	NO	Retrieve medication dispensing records for a patient	`GET /ws/fhir2/R4/MedicationDispense?patient`	read:clinical
`createMedicationDispense`	YES	Record a medication dispensing event with quantity and instructions	`POST /ws/fhir2/R4/MedicationDispense`	write:medication

4.7 Ingestion & Population Tools

Tool Name	Write?	Description	API Backing	Permission
`ingestPatientPDF`	YES	Parse a patient record PDF and ingest all entities into OpenMRS	`Multi-step: encounter + obs + allergy + condition + order`	write:all
`ingestFromHealthGorilla`	YES	Retrieve patient + conditions from Health Gorilla and sync to OpenMRS	`HG FHIR + POST /ws/rest/v1/condition`	write:all
`countPatients`	NO	Return aggregate patient count with optional filters	`GET /ws/rest/v1/patient`	read:population
`countEncounters`	NO	Return encounter count for a date range or provider	`GET /ws/fhir2/R4/Encounter`	read:population
`countPatientsByCondition`	NO	Return patient count filtered by condition concept	`GET /ws/fhir2/R4/Condition`	read:population

© 2026 MedPilot. Proprietary and Confidential.

Tool Name	Write?	Description	API Backing	Permission
`resolveConceptUUID`	NO	Look up concept UUID by name — used internally before all write tools	`GET /ws/rest/v1/concept?q`	read:metadata
`createConcept`	YES	Create a new concept for non-coded conditions (Health Gorilla import)	`POST /ws/rest/v1/concept`	write:admin

5. Natural Language Capabilities

The following table defines all supported chat intents and example prompts MedPilot MUST understand and resolve. These map directly to tool calls in Section 4.

Intent	Example Prompts	Tools Invoked	Write?
`search_patient`	*"Find patient Maria Santos" / "Search for patient ID 10001YY"*	`searchPatients, resolvePatient`	Read Only
`create_patient`	*"Add patient John Doe, male, born 1990-05-12, Nairobi"*	`createPatient`	Yes — preview + confirm
`patient_summary`	*"Summarize this patient" / "Give me a full overview of patient ABC"*	`getPatientDemographics, getConditions, getMedications...`	Read Only
`get_observations`	*"Show last HbA1c" / "Show vitals" / "What is the patient's SpO2?"*	`getObservations`	Read Only
`add_observation`	*"Add BP reading 140/90" / "Record weight 85kg" / "Log temperature 37.2°C"*	`createObservation`	Yes
`update_observation`	*"Update the height reading to 172cm" / "Correct the weight entry"*	`updateObservation`	Yes
`delete_observation`	*"Remove the duplicate height entry"*	`deleteObservation`	Yes
`get_conditions`	*"What conditions does this patient have?" / "Show problem list"*	`getConditions`	Read Only
`add_condition`	*"Add diagnosis: Type 2 Diabetes, active, confirmed, onset 2020-01-01"*	`createCondition, resolveConceptUUID`	Yes
`update_condition`	*"Mark hypertension as inactive" / "Change condition status to resolved"*	`updateCondition`	Yes
`delete_condition`	*"Delete the Malaria condition from the record"*	`deleteCondition`	Yes
`get_allergies`	*"Show allergy list" / "What is the patient allergic to?"*	`getAllergies`	Read Only
`add_allergy`	*"Add allergy: Aspirin, severe reaction, rash"*	`createAllergy`	Yes
`update_allergy`	*"Change Aspirin reaction severity to moderate"*	`updateAllergy`	Yes
`delete_allergy`	*"Remove the Penicillin allergy"*	`deleteAllergy`	Yes
`get_medications`	*"What medications is the patient on?" / "Show active orders"*	`getMedications`	Read Only

© 2026 MedPilot. Proprietary and Confidential.

Intent	Example Prompts	Tools Invoked	Write?
`add_medication`	*"Prescribe Metformin 500mg oral twice daily for 30 days"*	`createMedication, resolveConceptUUID`	**Yes**
`stop_medication`	*"Stop the Metformin order" / "Discontinue Aspirin"*	`updateMedication`	**Yes**
`dispense_medication`	*"Record dispensing of 20 tablets of Paracetamol 500mg"*	`createMedicationDispense`	**Yes**
`create_encounter`	*"Start a new encounter for this patient at Kenyatta clinic"*	`createEncounter`	**Yes**
`ingest_pdf`	*"Upload and ingest this patient record PDF"*	`ingestPatientPDF (multi-tool pipeline)`	**Yes — batch confirm**
`sync_health_gorilla`	*"Sync patient John Doe from Health Gorilla"*	`ingestFromHealthGorilla`	**Yes**
`population_query`	*"How many diabetes patients were seen this week?"*	`countPatientsByCondition, countEncounters`	**Read Only**
`operational_query`	*"List patients needing follow-up" / "Show today's workload"*	`countEncounters, getEncounters`	**Read Only**

6. Write Action Safety & Deduplication

6.1 Confirmation Workflow

All write actions MUST follow a 5-step confirmation workflow. The preview MUST show the exact JSON payload fields that will be sent to the API.

```
┌──────────────────────────────────────────────────────────────┐
│  ⚠  Pending Write Action — Please Confirm                    │
├──────────────────────────────────────────────────────────────┤
│  Action:        Create Condition                             │
│  Patient:       Maria Santos (UUID: 076154fc-...)            │
│  Condition:     Type 2 Diabetes Mellitus                     │
│  CIEL UUID:     119481AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA         │
│  Clinical Status:   active                                   │
│  Verification:      confirmed                                │
│  Onset Date:    2020-01-01T00:00:00.000+0000                 │
│  API Target:    POST /ws/rest/v1/condition                   │
├──────────────────────────────────────────────────────────────┤
│  [  CONFIRM  ]                        [  CANCEL  ]           │
└──────────────────────────────────────────────────────────────┘
```

6.2 Deduplication Rules

Entity	Dedup Check	Behavior if Duplicate Found
Patient	Same given name + family name + birthdate	Warn user with existing patient details; require explicit override to proceed
Allergy	GET /patient/{uuid}/allergy; compare codedAllergen UUID	Skip silently with log; inform user 'allergy already exists'

Entity	Dedup Check	Behavior if Duplicate Found
Medication	GET /order?patient={uuid}; check display name + action='NEW'	Skip with log; inform user 'medication already active'
Condition	No automatic dedup — concept UUID may differ by source	Warn if same concept name already active; allow override
Observation	No dedup — multiple readings allowed per concept	Always create; user can delete specific readings

6.3 Delete Safety Rules

DELETE Operations — Special Policy

Delete operations are irreversible. MedPilot MUST apply heightened confirmation requirements:

- Display a RED-highlighted warning banner in the confirmation modal
- State explicitly: 'This action cannot be undone.'
- Require the user to type 'DELETE' or click a separate destructive confirm button
- Log the full pre-delete resource payload to the audit store before executing
- Write-protected roles (nurse, read-only) MUST NOT have access to any delete tools

7. Security, Compliance & Audit

7.1 RBAC — Updated Permission Matrix

Permission	Clinician	Nurse	Admin	Read-Only	
`read:patient / read:clinical / read:population`	✓	✓	✓	✓	
`write:patient (createPatient)`	✓	—	—	—	
`write:encounter`	✓	Supervised	—	—	
`write:observation (create + update)`	✓	✓	—	—	
`delete:observation`	✓	—	—	—	
`write:condition (create + update)`	✓	Supervised	—	—	
`delete:condition`	✓	—	—	—	
`write:allergy (create + update)`	✓	✓	—	—	
`delete:allergy`	✓	—	—	—	
`write:medication (create + update)`	✓	—	—	—	
`write:ingestion (PDF + HG)`	✓	Supervised	—	—	

© 2026 MedPilot. Proprietary and Confidential.

Permission	Clinician	Nurse	Admin	Read-Only	
`write:admin` (`createConcept`)	—	—	✓	—	

7.2 Audit Log Requirements

REQ-ID	Requirement	Priority	Category
SEC-AUD-01	Every agent action MUST be logged: user ID, timestamp, patient UUID, prompt, intent, tools called, API endpoints hit, request payloads (redacted PHI), and response status	MUST	Audit
SEC-AUD-02	All DELETE operations MUST log the complete pre-delete resource payload before execution	MUST	Audit
SEC-AUD-03	All failed confirmation attempts (user cancels) MUST be logged	MUST	Audit
SEC-AUD-04	Audit logs MUST be retained for ≥ 7 years per HIPAA requirements in a tamper-evident append-only store	MUST	Compliance
SEC-AUD-05	Audit logs MUST NOT store raw PHI — apply field-level redaction or encryption at rest	MUST	Privacy

8. Non-Functional Requirements

8.1 Performance SLAs

Operation	P95 Target	Notes
Patient search / resolve	< 2 seconds	Single REST lookup
Patient summary (full)	< 5 seconds	Multiple parallel FHIR calls
Single observation / allergy / condition query	< 3 seconds	Single resource GET
Write action preview generation	< 2 seconds	Payload construction + UUID lookups
Write action execution	< 3 seconds	POST/PUT/PATCH/DELETE + audit log
PDF ingestion (full pipeline)	< 30 seconds	Parse + N entity writes sequentially
Health Gorilla sync (per patient)	< 15 seconds	HG fetch + condition upload
Population count query	< 3 seconds	Aggregate across patient population
Maximum hard timeout (any operation)	10 seconds	Fail gracefully with user notification

8.2 Reliability

REQ-ID	Requirement	Priority	Category
NFR-REL-01	Exponential backoff retry (max 3) on all transient OpenMRS API failures	MUST	Reliability
NFR-REL-02	Circuit breaker on Health Gorilla and all external API dependencies	MUST	Reliability

© 2026 MedPilot. Proprietary and Confidential.

REQ-ID	Requirement	Priority	Category
NFR-REL-03	UUID resolution failures MUST abort the write pipeline with a clear user error — never proceed with null UUIDs	MUST	Safety
NFR-REL-04	System MUST handle partial PDF ingestion failures gracefully — log per-entity result without rolling back succeeded writes	SHOULD	Reliability

9. Acceptance Criteria

REQ-ID	Requirement	Priority	Category
AC-01	All 24 chat intent types resolve to the correct tool(s) with ≥ 90% accuracy on the canonical test suite	MUST	NLP
AC-02	createPatient successfully registers a patient in OpenMRS with a valid auto-generated identifier	MUST	Registration
AC-03	PDF ingestion pipeline correctly parses and writes all 5 entity types from the sample patient_record.pdf	MUST	Ingestion
AC-04	Health Gorilla sync correctly creates patients and conditions in OpenMRS for all CSV test patients	MUST	Integration
AC-05	All DELETE operations display a heightened warning and require destructive confirmation before execution	MUST	Safety
AC-06	Allergy and medication deduplication correctly skips existing records in all test cases	MUST	Data Quality
AC-07	PATCH operations correctly update condition status, allergy severity, and medication status in OpenMRS	MUST	Write
AC-08	All write actions are logged with full payload and metadata in the audit store	MUST	Audit
AC-09	RBAC prevents nurses and read-only users from accessing delete and admin tools	MUST	Security
AC-10	All P95 performance SLAs are met under 50 concurrent user load	MUST	Performance
AC-11	Zero hallucinated data — all clinical claims cite a source record UUID	MUST	Safety
AC-12	UUID resolution failures abort writes and surface a clear error — no null-UUID writes occur	MUST	Safety

10. Feature Roadmap

Phase	Name	Capabilities	Target
Phase 1	**Core Read & Query**	Patient queries, population analytics, evidence grounding, full read across all entities	GA Release
Phase 2	**Full CRUD Lifecycle**	Patient registration, create/update/delete for conditions, allergies, observations, medications, encounters	Q2 Post-GA

© 2026 MedPilot. Proprietary and Confidential.

Phase	Name	Capabilities	Target
Phase 3	**Data Ingestion**	PDF ingestion pipeline, Health Gorilla sync, CSV bulk import, concept auto-creation	Q3 Post-GA
Phase 4	**External Integrations**	FDB drug interactions, IMO terminology enrichment — all modular	Q4 Post-GA
Phase 5	**Proactive Intelligence**	Care gap alerts, protocol adherence monitoring, proactive task creation	Future

© 2026 MedPilot. Proprietary and Confidential.